

Value-Function Approximation: From Supervised Learning to Stochastic Fixed-Point Iteration

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

Overview	2
1 The Setting: A Regression Problem with No Labels	2
1.1 What we would do with an oracle	2
1.2 The substitution chain	2
1.3 Notation for the operator view	3
2 The Statistics of the Targets	3
2.1 Conditional means and bias	3
2.2 Variance: counting the random ingredients	4
2.3 The master table and the first-moment criterion	5
3 Monte Carlo Is Exactly Supervised Learning	5
3.1 The ERM contract	5
3.2 The exactness proof	6
3.3 What the proof consumed, and what TD lacks	6
4 Bootstrapping Is Stochastic Fixed-Point Iteration	7
4.1 Three objects, kept distinct	7
4.2 The operator sequence and the fixed-point reading	8
4.3 Why self-teaching is not circular	8
4.4 The two nonstationarities	9
4.5 The empirical witness that the analogy fails	9
4.6 ERM violation accounting	9
5 Consequences: What Any Approximation Method Must Tolerate	10
5.1 Demand 1: online, incremental learning	10
5.2 Demand 2: tolerance of moving targets, and the step-size dichotomy	10
5.3 Forward pointer: the two demands, engineered	11
Summary	11

Overview

Prediction with function approximation is usually introduced as supervised learning: each update supplies a training pair (S_t, U_t) , and any regression method can then be applied. This report makes that claim precise and then inverts it. The supervised framing is *exactly* correct for one method (Monte Carlo) and structurally misleading for every bootstrapping method. The correct primitive for bootstrapped prediction is *stochastic fixed-point iteration* on the Bellman operator; supervised learning is the special case in which the operator generating the labels is constant. Everything in this report (the bias and variance of the targets, the exactness proof for Monte Carlo, the precise location of the breakage for TD, and the resulting demands on any approximation method) is an unpacking of that one statement.

1 The Setting: A Regression Problem with No Labels

1.1 What we would do with an oracle

Fix a policy π on an MDP and let v_π be its value function. We approximate v_π by a parameterized, differentiable family $\hat{v}(\cdot, \mathbf{w})$, $\mathbf{w} \in \mathbb{R}^d$, with $d \ll |\mathcal{S}|$. If an oracle handed us $v_\pi(s)$ at any queried state, the problem would be ordinary regression with loss

$$\overline{\text{VE}}(\mathbf{w}) = \sum_s \mu(s) [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2 = \mathbb{E}_{S \sim \mu} [(v_\pi(S) - \hat{v}(S, \mathbf{w}))^2], \quad (1)$$

where μ is the on-policy state distribution, and the stochastic-gradient update would be the *oracle update*

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t). \quad (2)$$

This update *requires* the oracle: $v_\pi(S_t)$ appears in it explicitly. No oracle exists, for three independent reasons: (i) *storage*: one value per state is impossible at $|\mathcal{S}| \sim 10^{20}$; (ii) *sample inefficiency*: estimating each state's value separately requires many visits to that exact state; (iii) *no generalization*: per-state estimation transfers nothing between similar states, which is the capability approximation exists to provide.

1.2 The substitution chain

Every prediction method in this part of the book is the oracle update (2) with $v_\pi(S_t)$ replaced by a *computable stand-in* U_t :

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t). \quad (3)$$

Running π produces the stream $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$, from which the training pairs $\{(S_t, U_t)\}$ are assembled. The methods differ *only* in U_t :

$$\text{Monte Carlo: } U_t = G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}, \quad (4)$$

$$\text{TD(0): } U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t), \quad (5)$$

$$n\text{-step TD: } U_t = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w}_t), \quad (6)$$

$$\text{DP: } U_t(s) = \mathbb{E}_\pi [R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) \mid S_t = s]. \quad (7)$$

Computability: MC needs no model but waits until episode end; TD(0) needs no model and is computable after one step; n -step waits n steps; DP is computable immediately but requires the

model $p(s', r \mid s, a)$, since the expectation in (7) is a sum against that distribution. The sampled methods need only the ability to *draw* one transition; DP needs the *distribution* itself. That access gap is the entire content of “model-free.”

Insight: the stand-in never meets the oracle

U_t replaces $v_\pi(S_t)$ in the residual of (2); the two never appear together in an update. Every question about a method reduces to one question about its stand-in: *how does U_t relate, statistically, to the $v_\pi(S_t)$ it replaces?* The next section answers this exactly.

1.3 Notation for the operator view

Write value functions as vectors in $\mathbb{R}^{|S|}$, and let $r_\pi(s) = \mathbb{E}_\pi[R_{t+1} \mid S_t = s]$, $P_\pi(s, s') = \sum_a \pi(a \mid s) p(s' \mid s, a)$. The Bellman operator for π is the *affine* map

$$T_\pi v = r_\pi + \gamma P_\pi v, \quad v_\pi = T_\pi v_\pi \quad (\text{unique fixed point}). \quad (8)$$

Affinity, not linearity, will matter: T_π applied to a *difference* sheds its reward term,

$$T_\pi u - T_\pi v = \gamma P_\pi(u - v), \quad \text{hence} \quad T_\pi^n u - T_\pi^n v = \gamma^n P_\pi^n(u - v). \quad (9)$$

2 The Statistics of the Targets

All statements in this section condition on the current parameters $\mathbf{w}$ (equivalently $\mathbf{w}_t$, frozen) and on $S_t = s$, with the trajectory generated by π itself.

2.1 Conditional means and bias

Monte Carlo. By the definition of the value function,

$$\mathbb{E}[G_t \mid S_t = s] = v_\pi(s). \quad (10)$$

This is not a theorem: $v_\pi(s) := \mathbb{E}_\pi[G_t \mid S_t = s]$, so unbiasedness is the definition read backwards. The entire content sits in the subscript π : the expectation is under the measure induced by acting with π , and (10) holds *because the data was generated by that same π* . If the trajectory came from a behavior policy $b \neq \pi$, the empirical averages would estimate b -flavored quantities instead, and recovering (10) would require importance-sampling ratios $\prod_k \pi(A_k \mid S_k) / b(A_k \mid S_k)$. On-policy data is the silent hypothesis under every unbiasedness claim in this report.

TD(0). Taking the conditional expectation of (5) and recognizing the result as one application of (8) to the current estimate,

$$\mathbb{E}[U_t \mid S_t = s] = (T_\pi \hat{v}_{\mathbf{w}})(s). \quad (11)$$

The bias follows from $v_\pi = T_\pi v_\pi$ and the affine identity (9):

$$\text{Bias}(s) = (T_\pi \hat{v}_{\mathbf{w}})(s) - (T_\pi v_\pi)(s) = \gamma (P_\pi(\hat{v}_{\mathbf{w}} - v_\pi))(s). \quad (12)$$

Two readings. First, the bias is the γ -discounted *expected next-state error*: the reward term of T_π is correct in both and cancels; only the bootstrapped part contributes. Second, the bias vanishes iff the error $\hat{v}_{\mathbf{w}} - v_\pi$ is invisible to $P_\pi(\cdot \mid s)$, in particular when $\hat{v}_{\mathbf{w}} = v_\pi$. Under genuine approximation that condition is *unattainable*: generically v_π does not lie in the representable family at all, so TD’s

bias never fully vanishes. This is the FA-specific sharpening of the tabular statement “biased until converged.”

n -step TD. Identically,

$$\mathbb{E}[U_t \mid S_t = s] = (T_\pi^n \hat{v}_{\mathbf{w}})(s), \quad \text{Bias}(s) = \gamma^n (P_\pi^n(\hat{v}_{\mathbf{w}} - v_\pi))(s). \quad (13)$$

The bias *decays geometrically in n* and is smoothed by n steps of the transition kernel. As $n \rightarrow \infty$ the bias is annihilated and the target becomes G_t : the n -step family interpolates the bias away at rate γ^n . This is the quantitative half of the bias–variance dial; the variance half is next.

DP. The target (7) is the conditional expectation of the TD(0) target:

$$U_t^{\text{DP}}(s) = \mathbb{E}[U_t^{\text{TD}} \mid S_t = s] = (T_\pi \hat{v}_{\mathbf{w}})(s), \quad (14)$$

a deterministic function given $\mathbf{w}$ (no trajectory is sampled; states are swept). Hence DP has *the same bias as TD(0) and zero variance*: TD(0) is DP with the expectation replaced by a single draw of its integrand.

2.2 Variance: counting the random ingredients

The cleanest way to see the variance ordering is to count the random draws each target is a function of, conditional on $S_t = s$:

- G_t is a function of the *entire random suffix* $A_t, S_{t+1}, A_{t+1}, S_{t+2}, \dots$, that is, every action draw and every transition draw to termination, plus (episodic) the random length T itself: $O(T)$ random quantities. The rewards are how the randomness enters the sum; the sources are policy and transition stochasticity at every step.
- $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ is a function of exactly *two* draws: $A_t \sim \pi(\cdot \mid s)$ and $(R_{t+1}, S_{t+1}) \sim p(\cdot, \cdot \mid s, A_t)$. (Note $\hat{v}(S_{t+1}, \mathbf{w})$ is random *through* S_{t+1} even with $\mathbf{w}$ frozen; TD’s variance is small, not zero.)
- U^{DP} is a function of *no* draws: zero variance.

Expanding $\text{Var}(G_t \mid S_t = s)$ as a sum of per-reward variances $\sum_k \gamma^{2k} \text{Var}(R_{t+k+1} \mid S_t = s)$ plus cross-covariances confirms the picture: every future step contributes noise. The covariances are generically nonzero, since future rewards are dependent through the shared trajectory, but they have no fixed sign (an environment in which reward now depletes reward later gives negative terms); the structural conclusion, variance growing with the horizon, stands regardless.

Insight: the bias–variance trade in one equation

Decompose $G_t = R_{t+1} + \gamma G_{t+1}$ and apply the law of total variance to the tail:

$$\text{Var}(G_{t+1} \mid S_t = s) = \underbrace{\mathbb{E}[\text{Var}(G_{t+1} \mid S_{t+1}) \mid S_t = s]}_{\text{within-tail randomness}} + \underbrace{\text{Var}(\mathbb{E}[G_{t+1} \mid S_{t+1}] \mid S_t = s)}_{\text{randomness through } S_{t+1}}.$$

TD’s target replaces the random tail G_{t+1} by a deterministic function of S_{t+1} , the plug-in $\hat{v}(S_{t+1}, \mathbf{w})$. This *deletes the entire within-tail term* and retains only the randomness flowing through the one visited next state. The price: the plugged-in function is $\hat{v}_{\mathbf{w}}$, not $\mathbb{E}[G_{t+1} \mid S_{t+1}] = v_\pi(S_{t+1})$, so the deleted *variance* is exchanged for the tail’s *bias* (12). One substitution, both effects: this is the bias–variance trade of bootstrapping, stated as an identity rather than a slogan.

2.3 The master table and the first-moment criterion

Key result: statistics of the prediction targets

target U_t	$\mathbb{E}[U_t S_t = s]$	bias w.r.t. $v_\pi(s)$	variance
G_t	$v_\pi(s)$	0	full suffix, $O(T)$ draws
$R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$	$(T_\pi \hat{v}_{\mathbf{w}})(s)$	$\gamma (P_\pi(\hat{v}_{\mathbf{w}} - v_\pi))(s)$	one transition
$G_{t:t+n}$	$(T_\pi^n \hat{v}_{\mathbf{w}})(s)$	$\gamma^n (P_\pi^n(\hat{v}_{\mathbf{w}} - v_\pi))(s)$	n transitions
DP	$(T_\pi \hat{v}_{\mathbf{w}})(s)$, deterministic	same as TD(0)	0

Reading by rows: MC and TD differ on the *bias* axis; TD and DP differ on the *variance* axis (TD samples the expectation DP computes). The n -step family moves monotonically along one dial: bias shrinks as γ^n , variance grows with n ; MC is the $n \rightarrow \infty$ endpoint, TD(0) the $n = 1$ endpoint.

Insight: stationary noise versus a moving mean

The statistical quantity separating MC from the bootstrapped targets is the *first moment of the label as a function of training time*. For MC, $\mathbb{E}[U_t | S_t = s] = v_\pi(s)$ is constant in t : samples scatter *widely around a fixed mean*, a high-variance *stationary* target. For TD, $\mathbb{E}[U_t | S_t = s] = (T_\pi \hat{v}_{\mathbf{w}_t})(s)$ moves with t : samples scatter *narrowly around a drifting mean*, a low-variance *nonstationary* target. Conflating the two is an error of category: the first is noise (handled by averaging), the second is drift (defeated by averaging). Section 4 is entirely about the second.

Geometrically: over the state space, each method is trying, at each step, to fit a *surface*. MC's surface is v_π , fixed for the whole run. TD's surface at step t is $T_\pi \hat{v}_{\mathbf{w}_t}$, redrawn after every update, migrating (one hopes) toward the fixed v_π as the iterates improve. DP fits the same moving surface as TD but evaluates it exactly: *moving mean, no sampling noise*.

3 Monte Carlo Is Exactly Supervised Learning

3.1 The ERM contract

Classical empirical risk minimization solves $\min_f \mathbb{E}_{(X,Y) \sim P}[\ell(f(X), Y)]$ by minimizing $\frac{1}{n} \sum_i \ell(f(X_i), Y_i)$ on a dataset $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n$, under three load-bearing clauses:

1. **Data:** $\mathcal{D}$ is drawn i.i.d. from P and sits still; multiple passes are made over a static set.
2. **Distribution:** one fixed P generates all data (train and deployment alike).
3. **Target:** the regression function $f^*(x) = \mathbb{E}[Y | X = x]$ is a fixed object; learning aims at a stationary bullseye.

3.2 The exactness proof

Key result: gradient MC is unbiased SGD on $\overline{VE}$

Proposition 1. *Let $S_t \sim \mu$ (on-policy visitation), let G_t be the return from S_t under π , and define the sample update direction $g_t = (G_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$. Then*

$$\mathbb{E}[g_t] = -\frac{1}{2} \nabla_{\mathbf{w}} \overline{VE}(\mathbf{w}).$$

Proof. (1) Differentiate the objective (1):

$$\nabla_{\mathbf{w}} \overline{VE}(\mathbf{w}) = \sum_s \mu(s) \nabla_{\mathbf{w}} [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 = -2 \sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}),$$

the interchange of $\nabla_{\mathbf{w}}$ and the sum/expectation being licensed because neither μ nor v_{π} depends on $\mathbf{w}$. [Verify: trivial for finite $\mathcal{S}$ (linearity over a finite sum); for infinite $\mathcal{S}$, dominated convergence with an integrable bound on $\|\nabla_{\mathbf{w}} \hat{v}\|$.]

(2) Condition the sample direction on S_t (tower property):

$$\mathbb{E}[g_t | S_t] = (\mathbb{E}[G_t | S_t] - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) = (v_{\pi}(S_t) - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}),$$

using that G_t contains no $\mathbf{w}$ (so it factors past the gradient and the conditioning does not couple to the parameters) and $\mathbb{E}[G_t | S_t] = v_{\pi}(S_t)$ from (10).

(3) Take the outer expectation over $S_t \sim \mu$ and compare with step (1):

$$\mathbb{E}[g_t] = \sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = -\frac{1}{2} \nabla_{\mathbf{w}} \overline{VE}(\mathbf{w}). \quad \square$$

3.3 What the proof consumed, and what TD lacks

The proof used exactly four properties. Each is a checkbox; TD fails two.

- (a) U_t contains no $\mathbf{w}$. Used twice: once so that differentiating the per-sample loss $(U_t - \hat{v})^2$ produces no target-derivative term, once so the conditioning in step (2) does not couple to the parameters. **TD fails this:** its target (5) carries $\hat{v}(S_{t+1}, \mathbf{w})$. Differentiating the per-sample loss honestly produces an extra term $-\gamma \nabla_{\mathbf{w}} \hat{v}(S_{t+1}, \mathbf{w})$; the standard TD update drops it. This omission is precisely what the prefix *semi*-gradient records, and it is why TD's update is not the gradient of *any* objective.
- (b) $\mathbb{E}[U_t | S_t] = v_{\pi}(S_t)$. Holds for G_t by definition, given on-policy data. **TD fails this:** its conditional mean is $(T_{\pi} \hat{v}_{\mathbf{w}})(S_t) \neq v_{\pi}(S_t)$, the bias (12).
- (c) $S_t \sim \mu$. On-policy visitation makes the sampling distribution coincide with the weighting in (1); the data performs the μ -weighting automatically. (Both MC and on-policy TD have this; off-policy data is where *this* clause breaks, a Report 10 matter.)
- (d) μ independent of $\mathbf{w}$. Licenses the interchange in step (1). True throughout prediction; it is the clause that fails for policy-gradient methods, where the parameter moves the sampling distribution and a score-function term appears.

Two distinct failures, (a) and (b), at two distinct lines of the same proof. The supervised analogy for TD does not degrade gracefully; it breaks at identifiable steps, and the two breakages have different

names (semi-gradient; bias) and different consequences (no objective exists; the destination is not v_π).

For MC, conversely, the analogy is not an analogy. With $U_t = G_t$ the training pairs are fixed external labels with conditional mean equal to the regression function being sought; gradient MC is ERM on $\overline{\text{VE}}$, executed by SGD, and converges (Robbins–Monro step sizes, suitable regularity) to the $\overline{\text{VE}}$ -minimizer; under linear approximation, the μ -weighted projection of v_π onto the representable subspace (developed in Report 09).

Remark 1 (The honest blemish: correlation). *One ERM clause is violated even by MC: the pairs are not i.i.d. States arrive Markov-dependently, and within an episode the labels overlap mechanically: $G_t = R_{t+1} + \gamma G_{t+1}$, so consecutive pairs share reward tails. This breaks the i.i.d. bookkeeping of the classical analysis but not the conclusion: the state marginal converges to the stationary μ , ergodic averages replace i.i.d. laws of large numbers, and the stochastic-approximation convergence theory (Robbins–Monro with Markov noise) carries the result. Correlation degrades constants, not the fixed point. Its real teeth appear later, combined with high-capacity approximators (Section 5).*

4 Bootstrapping Is Stochastic Fixed-Point Iteration

4.1 Three objects, kept distinct

Conflating the following three objects is the single most common confusion in this chapter; every nonstationarity claim should be checked against which of them it concerns.

1. **The objective** $\overline{\text{VE}}(\mathbf{w})$ of (1): a fixed surface over weight space, carved by (μ, v_π) alone.
2. **The label-generating function** at step t : the map $s \mapsto \mathbb{E}[U_t \mid S_t = s]$, i.e. what the labels are samples of.
3. **The destination**: the point the iteration converges to, if it converges.

Insight: $\overline{\text{VE}}$ never moves

$\overline{\text{VE}}$ is built from μ (fixed: stationary visitation of the fixed π) and v_π (fixed: the value of the fixed π). Neither changes during learning, so the objective surface is static, for MC and for TD. What moves under bootstrapping is object 2, the labels. TD does not descend a moving objective; it fails to descend the fixed one. The perception “bootstrapping makes the loss surface move” mistakes the moving labels for a moving landscape; the landscape never moved. TD simply is not walking downhill on it.

The accounting across the three regimes:

	objective $\overline{\text{VE}}$	label-generating function	destination
MC, fixed π	fixed	fixed ($= v_\pi$)	fixed ($\arg \min \overline{\text{VE}}$)
TD, fixed π	fixed (not descended)	moves ($T_\pi \hat{v}_{\mathbf{w}_t}$)	fixed (TD fixed point)
GPI control	moves	moves (both causes)	moves

4.2 The operator sequence and the fixed-point reading

At fixed π , TD’s labels at step t are samples of $T_\pi \hat{v}_{\mathbf{w}_t}$. Across training, the label-generating functions form the sequence

$$T_\pi \hat{v}_{\mathbf{w}_0}, T_\pi \hat{v}_{\mathbf{w}_1}, T_\pi \hat{v}_{\mathbf{w}_2}, \dots \quad (15)$$

Each update rewrites the labels for every state, because the labels are manufactured from the iterate being updated. The student is also the teacher; each step grades itself against a curriculum it just revised.

The sequence (15) stops moving exactly when the iterate reproduces its own labels:

$$\text{labels stationary} \iff \hat{v}_{\mathbf{w}} \text{ solves the (sampled, weighted) equation } \hat{v} = T_\pi \hat{v}. \quad (16)$$

That is, at a fixed point of the operator generating the labels. (Under linear approximation the attainable version is the *projected* equation $\hat{v} = \Pi T_\pi \hat{v}$; the projection Π and the convergence theory are Report 10’s subject. The structural point needs no projection: “labels stop moving” and “fixed point reached” are the same sentence.)

Key result: the thesis

Bootstrapped prediction is *stochastic fixed-point iteration*: generate labels by applying an operator to the current iterate, fit one step toward them, regenerate. Supervised learning is the special case in which the label-generating operator is *constant*; for Monte Carlo it maps every iterate to the same function v_π , so the labels never move and the iteration degenerates to honest regression. The book’s framing (“prediction is supervised learning, reuse existing methods”) is the constant-operator corner promoted to the general picture; the general picture is the fixed-point one, and every pathology and every demand in the remainder of this report is a property of operator *non-constancy*.

Within a single step, the supervised reading is exactly right: freeze $\mathbf{w}_t$, compute U_t as a number, take one regression step toward a frozen label. Across steps it is exactly wrong: the labels are re-manufactured from the updated iterate, and that regeneration loop, absent from any supervised problem, *is* the fixed-point iteration. “Per step supervised, across steps fixed-point” is the complete resolution of the analogy.

4.3 Why self-teaching is not circular

If the labels come from the model being fit, what anchors the process to reality? The reward. The operator is affine, $T_\pi v = r_\pi + \gamma P_\pi v$: every label contains *one step of real, externally supplied reward* plus a discounted recycled estimate. Iterating exposes the anchor:

$$T_\pi^k v_0 = \sum_{i=0}^{k-1} \gamma^i P_\pi^i r_\pi + \gamma^k P_\pi^k v_0 \xrightarrow{k \rightarrow \infty} v_\pi,$$

the recycled term decaying geometrically while truth accumulates one reward layer per application. Delete r_π and the iteration $v = \gamma P_\pi v$ collapses to the zero function: nothing external, nothing to learn. The bootstrap is a mechanism for *propagating* one step of ground truth per update through the self-referential estimate; circularity is avoided by exactly that one external ingredient per label.

4.4 The two nonstationarities

The word “nonstationary” is used for two different phenomena in this chapter. They have different causes, different lifetimes, and different objects in motion; the report keeps them separate.

Nonstationarity A: bootstrapping at fixed π (moving labels, fixed destination). The true target v_π is a constant: fixed policy, fixed value function, fixed bullseye. What moves is the *label* written down for each example, because the label is computed from the current iterate: at step 5 a state s might be labeled 7.2; at step 500, after $\mathbf{w}$ has changed, the same state is labeled 8.9. The label sequence is (15), marching (when all goes well) toward the fixed point. This nonstationarity is *self-inflicted*, produced by the algorithm’s own bootstrapping, and *transient*: it extinguishes itself as the iterates settle, by (16). Fixed destination, moving labels en route.

Nonstationarity B: GPI control (moving destination). Now π itself changes, $\pi_0 \rightarrow \pi_1 \rightarrow \pi_2 \rightarrow \dots$, and *three* things drift, in a causal chain: policy improvement moves π_k ; therefore the ground truth being estimated moves, $q_{\pi_0} \rightarrow q_{\pi_1} \rightarrow \dots$ (the bullseye itself is dragged); therefore the visitation distribution μ_{π_k} moves with it (the weighting under every expectation, and the norm in which error is measured, drifts); and the bootstrapped labels move for both reasons at once, stacked on top of nonstationarity A. This nonstationarity is *externally driven*, by the improvement step, and *persistent*: it extinguishes only when the policy stops improving, at the optimum.

Insight: the diagnostic question

To classify a nonstationarity, ask: *is the destination fixed?* If yes (fixed π , bootstrapping), only the labels move, the motion is transient, and the process is a fixed-point iteration settling onto a stationary solution. If no (GPI), the truth and the data distribution move together, the motion persists, and the estimator must *track* indefinitely rather than converge once. The step-size consequences of the two cases are the subject of Section 5.

4.5 The empirical witness that the analogy fails

A claim of the form “TD is just regression” admits refutation by exhibiting behavior impossible for regression. Least squares on fixed labels minimizes a fixed convex quadratic; with any sane step size the iterates are bounded and convergent. *Divergence is impossible*, because every step decreases (in expectation, eventually) a fixed nonnegative objective. Semi-gradient TD with linear approximation can *diverge to infinity* under off-policy sampling (Baird’s counterexample). No fixed objective exists whose descent could forbid it; that is consequence (a) of Section 3.3 made visible. The existence of the divergence is the proof that bootstrapped TD is not regression in any disguise; the conditions under which the fixed-point iteration nonetheless converges (on-policy weighting, contraction of the projected operator) are Report 10’s theorems.

4.6 ERM violation accounting

The three clauses of Section 3 are broken by three *different* culprits, one each:

ERM clause	status in prediction	culprit
1. i.i.d. static data	broken always	Markov correlation; for MC also overlapping label tails, $G_t = R_{t+1} + \gamma G_{t+1}$
2. fixed distribution	holds at fixed π (stationary μ)	broken only under GPI: μ_{π_k} drifts
3. fixed target function	holds for MC	broken only under bootstrapping: $T_\pi \hat{v}_{\mathbf{w}_t}$ moves; GPI breaks it twice (the truth q_{π_k} moves as well)

Three clauses, three different culprits: correlation is structural and always present; distribution drift is a control phenomenon; target motion is a bootstrapping phenomenon. A method’s suitability for RL is exactly the question of which of these violations it can absorb.

5 Consequences: What Any Approximation Method Must Tolerate

The data-stream structure of Section 1 and the label motion of Section 4 *derive* the two demands the book states; neither is an axiom.

5.1 Demand 1: online, incremental learning

The data is not a set; it is a stream. Pairs arrive one transition at a time, correlated, with no static collection to sweep repeatedly. Methods requiring the whole sample in hand (closed-form solves, batch tree induction, multi-epoch training over a frozen dataset) have nothing to hold. The method must improve from each example as it arrives: $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t(\cdots)$, the stochastic-approximation form.

Precision about the word *online* in the prediction setting: it qualifies the *value-estimate updates*, which are per-transition, during interaction, with no waiting for episode end (essential for continuing tasks, where there is no end to wait for). It does *not* mean the agent is learning how to act: π is fixed and given; behavior is not being adapted. And it is orthogonal to on-policy/off-policy, which concerns whose behavior generated the data, not when updates occur.

5.2 Demand 2: tolerance of moving targets, and the step-size dichotomy

For a *stationary* target, the Robbins–Monro schedule

$$\sum_t \alpha_t = \infty, \quad \sum_t \alpha_t^2 < \infty \quad (\text{e.g. } \alpha_t = 1/t) \quad (17)$$

delivers exact convergence: the first condition preserves the ability to travel any required distance; the second forces the sampling noise injected per step to be summable, so the iterate’s fluctuation is damped to zero. Against a *moving* target the second condition is the liability: once $\alpha_t \approx 0$ the iterate is frozen and cannot follow subsequent drift. Tracking requires steps bounded away from zero. With constant α , the tabular update unrolls to an exponential recency-weighted average of the targets,

$$V_t = (1 - \alpha)^t V_0 + \alpha \sum_{k < t} (1 - \alpha)^{t-1-k} U_k, \quad (18)$$

recent labels dominating, stale ones decaying geometrically: precisely the right estimator when old labels are out of date. The trade is explicit and unavoidable. Constant α buys trackability and pays with a persistent residual fluctuation of order $O(\alpha)$ around the (current) solution; Robbins–Monro buys exactness and pays with the inability to track. A method that can only converge onto a fixed objective cannot make this trade; that is the precise content of “less suitable for reinforcement learning.”

Why the machinery is needed even at fixed π , where nonstationarity A self-extinguishes: *early*, the labels $T_\pi \hat{v}_{\mathbf{w}_t}$ are grossly wrong and moving fast; a $1/t$ schedule over-commits to that early garbage (large steps exactly when labels are worst, vanishing steps by the time labels are good). And under nonstationarity B the motion never extinguishes at all. Constant or slowly-decaying steps are therefore the default throughout approximate RL, with the $O(\alpha)$ ball accepted as the cost of doing business against drift.

5.3 Forward pointer: the two demands, engineered

Deep Q-learning’s two signature components are direct answers to the two clauses this chapter identifies, in this chapter’s vocabulary. An *experience-replay buffer* samples past transitions in shuffled mini-batches, attacking clause 1 (decorrelating the stream toward the i.i.d. contract). A *target network* freezes a copy $\mathbf{w}^-$ of the parameters for many steps so the label-generating function $s' \mapsto \max_a \hat{q}(s', a; \mathbf{w}^-)$ is held piecewise-constant, attacking clause 3 (restoring a locally stationary target between synchronizations). Read this way, the DQN apparatus is an engineering program with one goal: drag the fixed-point iteration back toward the supervised contract for long enough stretches that large multi-pass approximators can be trained inside each stretch. The analysis of this report predicts the architecture.

Summary

One structural question governs the chapter: *does the target U_t contain the parameters?* If no (Monte Carlo), the labels are fixed external samples of the regression function v_π , the supervised analogy is a theorem (Section 3), and SGD descends the fixed objective $\overline{\text{VE}}$ to its minimizer. If yes (TD, n -step, DP), the labels are manufactured by an operator applied to the current iterate: the procedure is stochastic fixed-point iteration, supervised per step and self-referential across steps, anchored to reality by one layer of real reward per label, biased by $\gamma^n P_\pi^n(\hat{v} - v_\pi)$, variance-reduced by deleting the random tail, convergent exactly when the labels stop moving (16), and capable of failures (divergence) that no regression can exhibit. The criterion is graded, not binary: n interpolates the bias away at rate γ^n while accumulating variance, with MC as the constant-operator endpoint. The two nonstationarities are kept distinct throughout: moving labels with a fixed destination (bootstrapping, transient, self-extinguishing) versus a moving destination with a moving data distribution (GPI, persistent). The practical demands, incremental updates and drift-tolerant step sizes, are consequences rather than assumptions, and the canonical deep-RL architecture is the engineering of those consequences.